

0420 - AI Araştırma ve Geliştirme İş Paketleri

Alan	Değer
Doküman ID	0420
Proje	GeoLens Platform
Versiyon	1.0
Durum	Draft
Sahip	U2 AI Studio · Product
Tarih	29 Temmuz 2026
İlişkili	Bu doküman bağımsızdır. İçinde atıf yapılan tüm kavramlar aşağıda açıklanmıştır.

1. Giriş

Bu doküman, GeoLens platformuna katılan AI araştırmacısının görev tanımını, beklenen çıktıları, bu görevlerin neden verildiğini ve çıktıların ürünün hangi parçasında kullanılacağını tanımlar.

GeoLens'in mevcut AI/ML bileşenleri **tamamen kural tabanlı (rule-based) ve heuristic** seviyededir. Bu, MVP için yeterli olmakla birlikte, rekabet avantajı, ölçeklenebilirlik ve doğruluk açısından sürdürülebilir değildir. Araştırmacıdan beklenen, bu bileşenleri **makine öğrenimi / derin öğrenme tabanlı**, doğrulanabilir ve patentlenebilir algoritmalara dönüştürmektir.

Dil stratejisi: GeoLens küresel bir üründür. Tüm ML modelleri **en az Türkçe + İngilizce** desteklemeli, mimari olarak yeni dil eklemeye açık (language-agnostic) tasarlanmalıdır. Model seçiminde çok dilli modeller (XLM-R, mBERT, GPT-multilingual vb.) tercih edilmelidir. Dil bazlı fine-tuning ayrı branch'lerde değil, aynı modelde dil embedding'i ile yapılmalıdır.

2. Genel Beklentiler

Beklenti	Açıklama
Araştırma -> Ürün	Her araştırma sorusu, doğrudan bir ürün özelliğine veya kod dosyasına dönüşmelidir. Salt akademik çıktı kabul edilmez.
Ölçülebilir çıktı	Her iş paketi, tanımlı başarı kriterine sahiptir. "Oldu bitti" değil, "şu metrik bu değere ulaştı" gerekir.

Beklenti	Açıklama
Dil-agnostik tasarım	Tüm modeller en az TR + EN desteklemeli, yeni dil eklemek bir konfigürasyon değişikliğinden ibaret olmalıdır. Dile özel kod yazılmaz, dil embedding'i kullanılır.
Dokümantasyon	Her algoritma, açıklanabilirlik ve tekrarlanabilirlik ilkelerine uygun şekilde dokümente edilmelidir. Algoritmanın adımları, girdi/çıkış şeması, doğruluk metrikleri ve bağımlılıkları net olmalıdır.
Patentlenebilirlik	Geliştirilen algoritmaların patentlenebilir yönleri belirlenmeli ve İP-09'a girdi sağlanmalıdır.
Gold Dataset ile doğrulama	Tüm ML modelleri, İP-02'de oluşturulacak altın standart veri seti ile doğrulanmalıdır.

3. İş Paketleri (Öncelik Sırasına Göre)

İP-01: Prompt Taksonomisi ve Sınıflandırma Modeli

Alan	Detay
Süre	3 hafta
Öncelik	★★★★★ Kritik
Bağımlılık	—

Ne İsteniyor?

AI motorlarına gönderilen prompt'ların **intent**, **topic**, **persona**, **funnel** bazında sınıflandırıldığı bir taksonomi ve bu sınıflandırmayı otomatik yapan bir model.

Somut çıktılar:

- Prompt Taxonomy** (yeni bir doküman — taksonomi tanımı)
 - Intent sınıfları: presence, comparison, recommendation, category, problem + alt türler
 - Topic sınıfları: ürün, hizmet, marka, sektör, teknoloji + hiyerarşi
 - Persona sınıfları: tüketici, uzman, gazeteci, yatırımcı, öğrenci
 - Funnel aşamaları: farkındalık, değerlendirme, karar, satın alma
- 1000 adet etiketlenmiş örnek prompt** (CSV/JSON)
 - En az Türkçe + İngilizce, mimari olarak 10+ dile genişletilebilir
 - Her prompt için dil etiketi (lang: tr/en/de/fr/es/...) zorunlu
 - 5 sektör × 5 intent × 5 topic × 4 persona × 2 funnel çapraz dağılımı
 - Her prompt: intent, topic, persona, funnel, sektör etiketleri
- Prompt sınıflandırma modeli** (Python prototipi)
 - Intent/topic/persona/funnel sınıflandırması için ML modeli

- Doğruluk hedefi: > %85 F1

Neden Bu İş Veriliyor?

Şu an tüm prompt'lar elle yazılıyor ve hangi amaçla (intent) sorulduğu bilinmiyor. Bu, skorlama algoritmasının prompt türüne göre ağırlıklandırma yapamamasına yol açıyor. Örneğin "X markasını önerir misin?" ile "X markası hakkında bilgi ver" aynı ağırlıkta değerlendiriliyor. Bu taksonomi olmadan intent tabanlı ağırlıklandırma, sınıflandırma modeli ve anlamlı raporlama mümkün değildir.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Prompt Taxonomy	Prompt sınıflandırma modeli, bir prompt geldiğinde onu intent/topic/persona/funnel kategorisine atar. Sınıflandırma sonucu Visibility Index hesaplamasında farklı ağırlıklandırma yapmak için kullanılır (ör. recommendation intent'i presence intent'inden daha yüksek ağırlıklı).	Girdi: "Acme'nin en iyi rakibi kim?" → Çıktı: intent: comparison, topic: competitor, persona: consumer, funnel: evaluation → Bu etiketle Comparison intent'i %30 ağırlık alır, diğer intent'ler varsayılan ağırlıkta kalır.	Ölçüm motoru prompt'un amacını bilir, skor intent'e göre ağırlıklandırılır
1000 örnek prompt	Tüm ML modellerinin (sentiment, NER, hallüsinasyon) eğitimi ve testi için ortak referans verisi olarak kullanılır. Her model aynı prompt setiyle test edilir, böylece modeller arası karşılaştırma objektif olur.	prompt_001: {"text": "Acme hakkında ne düşünüyorsun?", "intent": "presence", "topic": "brand", ...} → Bu prompt sentiment modeline girdi olur, çıkan sentiment skoru gold dataset'teki etiketle karşılaştırılır.	Tüm testler aynı veriyle çalışır, "benim bilgisayarımda çalışıyordu" problemi kalkar
Sınıflandırma modeli	Yeni bir ölçüm başlatıldığında, prompt önce bu modele gönderilir. Model intent/topic/persona/funnel döndürür. Bu etiketler ölçüm sonucuna eklenir ve raporda gösterilir. API çağrısı: GET /v1/measure?prompt=... → arka planda classifyPrompt(prompt) çağrılır.	Gönderilen: POST /v1/measure { prompt: "Acme'nin fiyatları uygun mu?" } → Arka planda: classifyPrompt("Acme'nin fiyatları uygun mu?") → {intent: "comparison", confidence: 0.92} → Bu etiket measurement_jobs kaydına eklenir, skor hesaplamada kullanılır.	Gelen prompt otomatik sınıflandırılır, elle etiketleme ihtiyacı kalkar

İP-02: Altın Standart Veri Seti (Gold Standard Dataset)

Alan	Detay
Süre	4 hafta

Alan	Detay
Öncelik	★★★★★ Kritik
Bağımlılık	İP-01 (Prompt seti hazır olmalı)

Ne İsteniyor?

Tüm ML modellerinin eğitimi ve doğrulaması için **referans veri seti**. Her prompt için AI motor cevabı, mention, citation, entity, sentiment, hallucination etiketlerinin manuel olarak işaretlendiği bir veri seti.

Somut çıktılar:

1. Gold Dataset v1 (JSON Lines formatında)

- 500+ prompt için manuel etiketlenmiş referans verisi
- En az TR + EN dengeli dağılım (250 TR, 250 EN)
- Her prompt için: 1 lang , beklenen cevap özeti, mention listesi (marka/ürün/rakip), citation listesi (URL + tür), entity listesi (tip + değer), recommendation var/yok, sentiment (pozitif/nötr/negatif)
- Train/test split: %80 / %20
- Etiketleyiciler arası uyum (inter-annotator agreement) > %90

2. Annotation Guide (yeni bir doküman — etiketleme kılavuzu)

- Adım adım etiketleme talimatı
- Karar ağaçları: "Bu mention mu?" → "Evet ise marka mı, ürün mü?"
- Dil bazında etiketleme farklılıkları (İngilizce büyük harf duyarlılığı, Türkçe karakter sorunları)
- Sık yapılan hatalar ve örnekleri
- 10 örnek üzerinde adım adım gösterim (5 TR, 5 EN)

3. Labeling Rules

- Mention tespit kuralları: tam eşleşme, kısmi eşleşme, eş anlamlı, yanlış pozitif
- Citation sınıflandırma: direct/attribution/directional
- Entity tipi belirleme: bağlam bazlı karar matrisi
- Sentiment: keyword + bağlam tabanlı

Neden Bu İş Veriliyor?

Bugün sentiment analizi 20 kelimelik bir listeyle çalışıyor. Hallüsinasyon tespiti 5 heuristic kuraldan ibaret. Hiçbir ML modeli eğitilemiyor çünkü **eğitim verisi yok**. Bu veri seti olmadan sonraki tüm ML çalışmaları (sentiment, NER, hallüsinasyon) temelsiz kalır. Bu nedenle ilk adım, bilimsel geçerliliği olan bir referans veri seti oluşturmaktır.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Gold Dataset	Ölçüm sonucu gelen AI cevapları, gold dataset'teki referans cevaplarla karşılaştırılır. Her algoritma değişikliğinde önce gold dataset üzerinde test koşulur , eski algorithmadan	Test senaryosu: Gold dataset'teki prompt_042'nun beklenen sentiment değeri <code>positive(0.85)</code> . Yeni sentiment modeli bu prompt için <code>0.82</code> döndürürse → kabul edilir (>0.80 eşik). <code>0.45</code> döndürürse →	Algoritma değişikliklerinde objektif karşılaştırma — "eskisinden iyi mi?" sorusu veriyile cevaplanır

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
	beter olmadığı doğrulanır. CI/CD pipeline'ında otomatik test olarak çalışır.	pipeline kırmızı, algoritma reddedilir.	
Annotation Guide	Yeni prompt'lar eklendikçe bu kılavuza göre etiketlenir. Inter-annotator agreement hesaplanır: iki farklı kişi aynı prompt'u etiketler, uyum oranı > %90 olmalıdır.	Kılavuzdaki kural: "Eğer AI cevabında marka adı geçiyorsa ve olumlu sıfatla birlikte kullanılıyorsa → sentiment: positive ". Örnek: "Acme kaliteli ürünler sunar" → mention: ["Acme"], sentiment: positive	Veri seti büyürken tutarlılık korunur
Labeling Rules	AI cevabı ayrıştırılırken hangi kelimenin marka, hangisinin ürün olduğuna bu kurallarla karar verilir. Karar ağacı şeklinde çalışır: "Bu kelime büyük harfle başlıyor mu? → Evet → Şirket adı mı? → ..."	Kural: "Bir kelime hem marka hem ürün olabilir (ör. Apple). Bağlamda 'iPhone' geçiyorsa → Apple product. 'şirket' geçiyorsa → Apple brand." Input: "Apple'ın yeni MacBook'u çıktı" → Apple: brand, MacBook: product	"Bu marka mı, ürün mü?" karışıklığı önlenir

İP-03: Transformer Tabanlı Duygu Analizi (Sentiment Analysis)

Alan	Detay
Süre	4 hafta
Öncelik	★★★★★ Kritik
Bağımlılık	İP-02 (Gold dataset hazır olmalı)

Ne İsteniyor?

Mevcut ~20 kelimelik keyword listesini, **fine-tune edilmiş çok dilli transformer modeli** (XLM-R, mBERT veya benzeri) ile değiştirmek. Bağlam farkındalığı olan, ironi ve dolaylı ifadeleri de yakalayabilen bir sentiment sınıflandırıcı. Model aynı anda en az Türkçe ve İngilizce desteklemeli, yeni dil eklemek ek fine-tuning ile mümkün olmalıdır.

Somut çıktılar:

- Çok dilli transformer modeli** (XLM-R, mBERT, GPT-multilingual veya benzeri)
 - En az Türkçe + İngilizce, yeni diller eklenebilir mimaride
 - Sınıflar: positive, neutral, negative, mixed
 - Gold dataset üzerinde validation
 - Hedef: > %90 F1 (mevcut ~%60-70 seviyesinden)
- Model serving kodu** (Python + ONNX veya benzeri)
 - Go servisinden çağrılabilir API (REST veya gRPC)
 - Inference süresi: cevap başına < 200ms

- Batch processing desteği

3. Performans karşılaştırma raporu

- Eski (keyword) vs yeni (transformer) model karşılaştırması
- Her sınıf için precision, recall, F1
- 10 edge case analizi: ironi, karşılaştırma, dolaylı ifade, sektörel jargon

Neden Bu İş Veriliyor?

Mevcut sentiment analizi `positiveWords` ve `negativeWords` diye iki Go slice'ından ibaret (`internal/sentiment/engine.go:129-134`). İngilizce ve Türkçe karışık ~20 kelimeyi sayıp oranlıyor. Bağlamı anlamıyor: İngilizce "This product is not bad" cümlesini `bad` kelimesini görünce negatif sayıyor. Türkçe "Bu ürün kötü değil" cümlesini de aynı şekilde. Bu seviyedeki bir analiz, müşteriye sunulamayacak kadar hatalıdır. Transformer tabanlı çok dilli bir modele geçiş, bağlam farkındalığı ve doğruluk açısından zorunludur.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Fine-tune model	Her ölçüm sonucunda AI cevabı bu modele gönderilir. Model <code>lang</code> parametresine göre doğru dilde inference yapar. Eski sistemde İngilizce "not bad at all" cümlesi <code>bad</code> kelimesini görünce negatif sayardı. Türkçe "Bu ürün kötü değil" de aynı hatayı alırdı. Yeni model bağlamı anlayarak her iki dilde de doğru sınıflar.	İngilizce girdi: "Acme products are not bad at all, actually quite reliable" → Eski: negative (0.40) → Yeni: positive (0.85), <code>lang: en</code> Türkçe girdi: "Acme'nin ürünleri kötü değil, hatta oldukça başarılı" → Eski: negative (0.50) → Yeni: positive (0.82), <code>lang: tr</code> → <code>GET /v1/sentiment?brand=acme</code> endpoint'inde dil bazlı kırımım gösterilir.	Keyword listesi yerine bağlam bilen çok dilli model — olumsuzluk ekini doğru anlar
Model serving API	Python'da çalışan model, bir REST API olarak ayağa kaldırılır. Go backend'deki <code>AnalyzeSentiment()</code> fonksiyonu bu API'ye <code>POST /predict</code> çağrısı yapar. İstekte <code>lang</code> alanı zorunludur. Model dil bazlı embedding seçerek inference yapar. Inference süresi < 200ms olmalıdır.	Türkçe çağrı: <code>POST /predict {"text": "Acme harika bir şirket", "lang": "tr"} → Dönen:</code> <code>{"sentiment": "positive", "confidence": 0.94}</code> İngilizce çağrı: <code>POST /predict {"text": "Acme is a great company", "lang": "en"} → Dönen:</code> <code>{"sentiment": "positive", "confidence": 0.92}</code> Her iki sonuç <code>analysis.sentiment_scores</code> tablosuna <code>lang</code> alanıyla birlikte yazılır, dashboard'da dil filtresiyle gösterilir.	Go backend'den çok dilli Python modeline inference çağrısı
Karşılaştırma raporu	Rapor, eski keyword modeli ile yeni transformer modelinin aynı 1000 prompt üzerindeki	Rapordan: TR (500 prompt): Eski F1=0.58 → Yeni F1=0.91 (+%33)	"Neden çok dilli ML modele geçtik?"

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
	sonuçlarını dil bazında kırılımla karşılaştırır. Her dil için ayrı F1 skoru gösterir. Hangi dillerde modelin iyi, hangilerinde zayıf olduğu görülür.	EN (500 prompt): Eski F1=0.62 → Yeni F1=0.93 (+%31) Edge case (TR): "Acme rakiplerine göre daha mı pahalı?" → Eski: negative (0.35) (çünkü "pahalı" negatif listede) → Yeni: neutral (0.52) (soru cümlesi) → Doğru: neutral Edge case (EN): "Is Acme more expensive than competitors?" → Eski: negative (0.30) (çünkü "expensive" negatif listede) → Yeni: neutral (0.55) (soru cümlesi) → Doğru: neutral	sorusunun verili cevabı

İP-04: Hallüsinasyon Tespiti — Cross-Source Validation

Alan	Detay
Süre	4 hafta
Öncelik	★★★★★ Kritik
Bağımlılık	İP-02 (Gold dataset)

Ne İsteniyor?

Mevcut 5 heuristic kuralı (T1-T5) yerine, **birden çok AI motoru ve kaynağı çapraz referanslayarak** tutarsızlıkları tespit eden bir sistem.

Somut çıktılar:

1. Cross-source validation algoritması

- Aynı prompt'un farklı motorlardaki cevaplarını karşılaştırma
- Tutarsızlık tespiti: "ChatGPT X dedi, Gemini Y dedi — hangisi doğru?"
- Kaynak doğrulama: citation'daki URL gerçekten o bilgiyi içeriyor mu?
- Confidence skoru: her tespit için güven oranı (0-1)

2. LLM-as-Judge pipeline

- Bir LLM'a (ör. Claude) diğer LLM'ların cevaplarını değerlendirtme
- Değerlendirme kriterleri: factual consistency, source attribution, contradiction rate
- Maliyet optimizasyonu: ne sıklıkla judge çağrılmalı?

3. Hallüsinasyon türleri (güncellenmiş T1-T5+)

- Mevcut T1-T5'in iyileştirilmesi
- Yeni türler: contradiction (çelişki), fabrication (uydurma), outdated (güncel olmayan bilgi)
- Her tür için: tespit yöntemi, severity, confidence hesaplama

Neden Bu İş Veriliyor?

Mevcut hallüsinasyon tespiti (`internal/sentiment/engine.go:263-326`) marka adı yanıtta geçiyor mu diye bakıyor, "kaynak" kelimesi varsa işaretliyor, sayı+% varsa "critical" etiketi basıyor. Bunların hiçbirisi gerçek hallüsinasyon tespiti değil — sadece şüpheli desen yakalama. Gerçek bir hallüsinasyon tespiti için çapraz kaynak doğrulama ve faktualite kontrolü gereklidir.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Cross-source validation	Aynı prompt 3 farklı AI motoruna (ChatGPT, Gemini, Perplexity) gönderilir. Gelen cevaplar karşılaştırılır: ChatGPT "Acme 2023'te kuruldu" derken Gemini "Acme 2020'de kuruldu" derse → tutarsızlık tespit edilir , hallucination_type: contradiction olarak kaydedilir. Citation'daki URL'ler ayrıca ziyaret edilerek bilginin gerçekten o URL'de olup olmadığı kontrol edilir.	3 motor cevabı: ChatGPT: "Acme 2023'te kuruldu [kaynak: acme.com]" → acme.com ziyaret: "2020'de kuruldu" yazıyor → Tutarsızlık: hallucination_type: T2 (fabricated citation), confidence: 0.85, severity: critical → Müşteri raporunda kırmızı uyarı olarak gösterilir.	Heuristic kurallar yerine çapraz doğrulama ile gerçek hallüsinasyon tespiti
LLM-as-Judge	Bir LLM (ör. Claude 4), diğer LLM'ların cevaplarını değerlendirir. Şu soruları sorar: "Bu cevap verilen kaynakla uyumlu mu?", "İç çelişki var mı?", "Rakamsal veri kaynaklı mı?" Judge çağırısı, belirli bir şüphe eşiği aşıldığında tetiklenir (maliyet optimizasyonu).	Judge girdisi: {"prompt": "Acme'nin pazar payı nedir?", "responses": [{"engine": "chatgpt", "text": "%25"}, {"engine": "gemini", "text": "%30"}]} → Judge çıktısı: {"contradiction": true, "factual_consistency": 0.3, "recommendation": "doğrulanmamış istatistik"} → Bu çıktı guardrail.evaluations tablosuna yazılır.	Otomatik doğruluk kontrolü, yanıltıcı verinin müşteriye gitmesi engellenir
Güncellenmiş türler	Hallüsinasyon tespitinde yeni türler (T5: contradiction, T6: fabrication, T7: outdated) eklenir. Her türün tespit yöntemi, severity seviyesi ve confidence hesaplama formülü belirlenir. Müşteri raporunda her tür farklı renk/simgelye gösterilir.	T5 (contradiction): "Acme 2020'de kuruldu, 2019'dan beri lider" → İki cümle çelişiyor → severity: high T6 (fabrication): "Acme'nin 5 milyon kullanıcısı var" ama hiçbir kaynak göstermiyor → severity: critical T7 (outdated): "Acme'nin CEO'su Ahmet Yılmaz" ama 2024'te değişti → severity: medium	Standart dokümanite edilmiş, her türün tespit yöntemi net

İP-05: Varlık Tanıma (Entity Recognition / NER)

Alan	Detay
Süre	4 hafta
Öncelik	★★★★☆ Yüksek
Bağımlılık	İP-02 (Gold dataset)

Ne İsteniyor?

AI cevaplarından **marka, ürün, rakip, teknoloji, organizasyon, lokasyon, kişi** varlıklarını otomatik çıkaran bir NER sistemi.

Somut çıktılar:

1. NER Model

- Çok dilli NER modeli (XLM-R, mBERT veya benzeri) — en az TR + EN
- 7+ entity tipi tanımlı
- Hedef: > %85 F1

2. Extraction Rules (hibrit yaklaşım)

- Regex tabanlı kurallar: marka adı desenleri, URL çıkarma
- NER modeli: bağlam bazlı entity tespiti
- LLM-as-judge: regex/NER yetmediğinde devreye giren fallback
- Post-processing: duplicate removal, çelişki çözümü, confidence scoring

3. Benchmark Dataset

- 200+ manuel etiketlenmiş entity içeren test seti (100 TR, 100 EN)
- Her test için: raw AI cevabı, lang etiketi, beklenen entity listesi
- Zor senaryolar: eş sesli markalar ("Apple" → şirket mi meyve mi?), yabancı dil entity'ler

Neden Bu İş Veriliyor?

Şu an AI cevaplarında "Apple" geçince bunun şirket mi, meyve mi olduğu ayırt edilemiyor. Marka mention'ları basit string match ile bulunuyor. Bu, Presence Share ve Competitor Context skorlarının hatalı hesaplanmasına yol açıyor. NER modeli olmadan entity bazlı analiz ve doğru skorlama mümkün değildir.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
NER Model	AI cevabı geldiğinde, lang alanına göre doğru dil embedding'i seçilir ve entity'ler çıkarılır. "Apple" entity'si çıkarıldığında, model bağlama göre brand veya product olarak etiketler. Dil farkı	EN girdi: "Apple's new MacBook Pro competes with Samsung's Galaxy Book" → NER: [{entity: "Apple", type: "brand"}, {entity: "MacBook Pro", type: "product"}, {entity: "Samsung", type: "competitor"}, {entity: "Galaxy Book", type:	"Apple"ın şirket mi ürün mü olduğu dil bağımsız ayırt edilir

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
	gözetmeksizin aynı model hem TR hem EN cevapları işler.	"product"]] TR girdi: "Apple'ın yeni MacBook Pro'su, Samsung'un Galaxy Book'una rakip oldu" → NER: aynı çıktı → Presence Share: Apple = 1, Competitor Context: Samsung	
Extraction Rules	NER modeli bir entity bulduğunda, Extraction Rules dil bağımsız kurallarla devreye girer. Regex kuralları Unicode karakterleri destekler (Türkçe ü/ğ/ş/ı, Almanca ß, Fransızca é/è vb.). NER emin değilse LLM-as-judge fallback'i aynı prompt'u entity'in dilinde gönderir.	TR belirsiz: "Yeni Apple ürünü çok beğenildi" → Regex: Büyük harf + ürün kelimesi → NER: confidence: 0.6 → LLM-as-judge (TR): "Apple burada bir markadır" EN belirsiz: "Apple released a new device" → NER: confidence: 0.65 → LLM-as-judge (EN): "Apple here is a brand" Her iki dilde aynı sonuç, aynı pipeline.	"Rakip mi, teknoloji mi, ürün mü?" karışıklığı tüm dillerde önlenir
Benchmark Dataset	NER modelinin doğruluğu her gün bu dataset ile dil bazında test edilir. TR ve EN için ayrı F1 skorları raporlanır. Dillerden biri eşiğin altına düşerse alarm üretilir. CI pipeline'ında otomatik çalışır.	EN test: "Google and Amazon lead the AI race" → [{Google: brand}, {Amazon: brand}] → F1: 0.98 TR test: "Google ve Amazon yapay zeka yarışında başı çekiyor" → [{Google: brand}, {Amazon: brand}] → F1: 0.95 Edge case (EN): "Is Apple healthy?" → Beklenen: [] (meyve) → Model: [] (doğru) Edge case (TR): "Elma sağlıklı mı?" → Beklenen: [] → Model: [] (doğru)	Entity çıkarım doğruluğu her dilde sürekli ölçülür

İP-06: Visibility Index Algoritma Optimizasyonu

Alan	Detay
Süre	3 hafta
Öncelik	★★★★★ Kritik
Bağımlılık	İP-02, İP-03, İP-05 (Veri seti + sentiment + NER hazır olmalı)

Ne İsteniyor?

Mevcut 4 bileşenli visibility skoru (Presence %35, Position %25, Source %20, Competitor %20) **matematiksel olarak optimize etmek** ve ağırlıkları veriye belirlemek.

Somut çıktılar:

1. Matematiksel Model (LaTeX/PDF)

- Genişletilmiş Visibility Index formülü: $VI = w_1 \cdot M + w_2 \cdot P + w_3 \cdot C + w_4 \cdot R + w_5 \cdot A + w_6 \cdot F + w_7 \cdot S$
- Her bileşenin alt formülü, normalizasyon yöntemi, güven aralığı
- Toplam skor 0-100, CI dinamik hesaplama (şu an ± 5 hardcoded)

2. Ağırlıklandırma Yöntemi

- AHP (Analytic Hierarchy Process) veya regresyon tabanlı ağırlık belirleme
- Duyarlılık analizi: ağırlık değişince skor nasıl etkilenir?
- Sektör bazlı profiller: e-ticaret $\rightarrow$ recommendation ağırlıklı, sağlık $\rightarrow$ authority ağırlıklı

3. Doğrulama Raporu

- Gold dataset üzerinde MAE, RMSE, R^2 , Spearman korelasyonu
- 10+ edge case testi
- Eski algoritma vs yeni algoritma karşılaştırması

Neden Bu İş Veriliyor?

Mevcut ağırlıklar (`internal/measure/service.go`) tamamen keyfi — "presence daha önemli, %35 verelim" mantığıyla belirlenmiş. Oysa visibility kavramı çok boyutlu: bir marka her yerde geçiyor olabilir ama hep olumsuz bağlamda (yüksek presence, düşük sentiment). Veya hiç geçmiyor olabilir ama geçtiği yerde tavsiye ediliyor (düşük presence, yüksek recommendation). Matematiksel model ve veriye belirlenmiş ağırlıklar olmadan skurun güvenilirliği ve açıklanabilirliği sağlanamaz.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Matematiksel Model	Ölçüm sonuçları hesaplanırken bu formül kullanılır. 7 bileşenin her biri (Mention, Position, Citation, Recommendation, Authority, Freshness, Confidence) ayrı ayrı hesaplanır, ağırlıklarıyla çarpılır ve toplanır. Formül <code>calculateScore()</code> fonksiyonunda kodlanır. Her hesaplama <code>calculation_runs</code> tablosuna kaydedilir (deterministik replay için).	Hesaplama: $VI = 0.25 \times 0.85 + 0.20 \times 0.60 + 0.15 \times 0.90 + 0.10 \times 0.70 + 0.10 \times 0.50 + 0.10 \times 0.80 + 0.10 \times 0.95 \rightarrow VI = 0.2125 + 0.12 + 0.135 + 0.07 + 0.05 + 0.08 + 0.095 = 0.7625 \rightarrow CI: \pm 3.2 \rightarrow \{value: 76, ci_low: 73, ci_high: 79\} \rightarrow$ Dashboard'da "76 $\pm$ 3" olarak gösterilir.	Visibility Index'in şeffaf, bilimsel formülü
Ağırlıklandırma	Her sektör için farklı ağırlık profili tanımlanır. Varsayılan profil tüm markalar için geçerlidir. Müşteri sektörünü seçtiğinde ilgili profil kullanılır. Ağırlıklar AHP veya regresyon ile belirlenir, sensitivity	Varsayılan: {M: 0.25, P: 0.20, C: 0.15, R: 0.10, A: 0.10, F: 0.10, S: 0.10} E-ticaret: {M: 0.20, P: 0.15, C: 0.10, R: 0.25, A: 0.05, F: 0.15, S: 0.10} (recommendation ağırlıklı)	Keyfi ağırlıklar yerine veriye belirlenmiş optimal ağırlıklar

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
	analysis raporuyla hangi ağırlığın skoru ne kadar etkilediği gösterilir.	Sağlık: {M: 0.15, P: 0.10, C: 0.20, R: 0.05, A: 0.30, F: 0.10, S: 0.10} (authority ağırlıklı) Sensitivity: Citation ağırlığı %10 artarsa → VI +2.3 puan değişir.	
Doğrulama Raporu	Yeni algoritma gold dataset üzerinde çalıştırılır, MAE/RMSE/R ² hesaplanır. Eski algoritmayla karşılaştırılır. Rapor, "Yeni algoritma eskiye göre %X daha doğru" kanıtını sağlar. Bu rapor olmadan algoritma değişikliği yapılmaz.	Rapordan: Gold dataset'te 500 prompt → Eski algoritma: MAE=8.2, RMSE=11.5, R ² =0.72 → Yeni algoritma: MAE=4.1, RMSE=6.2, R ² =0.89 → İyileşme: %50 daha az hata, %23 daha yüksek korelasyon → Edge case: "Sıfır mention" senaryosunda eski 0 verirken yeni 12 veriyor (manual assessment 15 → daha doğru).	Her algoritma değişikliğinde objektif validasyon

İP-07: Fırsat Motoru (Opportunity Engine)

Alan	Detay
Süre	3 hafta
Öncelik	★★★★☆ Yüksek
Bağımlılık	İP-06 (Yeni visibility index hazır olmalı)

Ne İsteniyor?

Müşteriye "skorun düşük" demek yerine "**citation kaynağını çeşitlendir, FAQ sayfana şu soruları ekle**" gibi somut aksiyon önerileri üreten ML tabanlı bir sistem.

Somut çıktılar:

1. Recommendation Model

- 10+ öneri kategorisi: visibility kaybı, citation açığı, içerik eksikliği, otorite zafiyeti, yapısal veri hatası, rakip tehdidi
- Her kategori: tespit koşulu, öneri metni şablonu, kanıt etiketi
- ML-based opportunity scoring (mevcut rule-based'in yerine)

2. Opportunity Scoring

- OpportunityScore = Impact × Urgency × Confidence
- Impact: müşterinin visibility'ine potansiyel etki (1-10)
- Urgency: ne kadar hızlı aksiyon alınmazsa kayıp büyür? (1-10)
- Confidence: tespitin doğruluk olasılığı (0-1)

- ML ile tahmin: geçmiş veriden hangi önerilerin gerçekten etkili olduğunu öğrenme

3. Rule Set (güncellenmiş)

- Mevcut 8 baz + 5 sektör kuralının iyileştirilmesi
- Veriyle doğrulanmış yeni kurallar
- Her kural için sektör filtresi ve etki/çaba skoru

Neden Bu İş Veriliyor?

Mevcut recommendation engine (`internal/recommendation/service.go`) statik kurallarla çalışıyor: eğer skor düştüyse "skorun düştü" diyor, citation azsa "citation'ı artır" diyor. Hangi önerinin gerçekten etkili olduğunu öğrenemiyor. ML tabanlı bir fırsat motoru, geçmiş veriden öğrenerek önerileri kişiselleştirebilir ve önceliklendirebilir. Bu olmadan öneriler genel geçer ve düşük etkili kalır.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Recommendation Model	Her ölçüm sonrası tetiklenir. ScoreSnapshot (güncel + önceki skor) ve AuditSnapshot (robots.txt, structured data, bot erişimi) alınır. Bu verilerle 10+ kural değerlendirilir. Her kural için $\text{Impact} \times \text{Urgency} \times \text{Confidence} = \text{OpportunityScore}$ hesaplanır. En yüksek skorlu öneriler müşteri dashboard'unda gösterilir.	Trigger: "Acme" markası için yeni ölçüm bitti → ScoreSnapshot: <code>{current: 42, previous: 68, drop: %38}</code> → Kural 1 tetiklendi: <code>score-drop</code> → <code>{impact: 9, urgency: 8, confidence: 0.95}</code> → OpportunityScore: 68.4 → Öneri üretildi: "Acme'nin AI görünürlük skoru son 30 günde %38 düştü. Rakiplerinizden biri sizi geçmiş olabilir. Detaylı rekabet analizi yapmanızı öneririz." → Dashboard'da kırmızı uyarı olarak gösterilir.	Müşteriye somut, önceliklendirilmiş aksiyon önerileri
Opportunity Scoring	Her öneriye $\text{impact} (1-10) \times \text{urgency} (1-10) \times \text{confidence} (0-1)$ formülüyle skor verilir. Öneriler bu skora göre sıralanır. Müşteri önce en yüksek etkili, en acil ve en güvenilir öneriyi görür. ML, geçmiş veriden hangi önerilerin gerçekten etkili olduğunu öğrenir ve zamanla ağırlıkları günceller.	Sıralı liste: 1. Citation çeşitlendirme → Score: 72 → <code>{impact: 8, urgency: 9, confidence: 0.90}</code> 2. FAQ sayfası ekle → Score: 45 → <code>{impact: 6, urgency: 5, confidence: 0.70}</code> 3. robots.txt düzelt → Score: 30 → <code>{impact: 7, urgency: 3, confidence: 0.60}</code> Müşteri önce "Citation çeşitlendirme" önerisini görür → "Harekete Geç" butonuna tıklayınca	Öneriler etki/çaba matrisinde sıralanır, müşteri önce en hızlı kazancı görür

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
		recommendation.results tablosunda applied=true olur.	
Rule Set (güncellenmiş)	8 varsayılan kural + 5 sektör kuralı, her ölçüm sonrası evaluateBrand() fonksiyonunda çalıştırılır. Her kuralın bir koşulu vardır: "skor %15+ düştü mü?", "citation domain sayısı < 3 mü?" gibi. Koşul true ise öneri üretilir. Öneri metni müşterinin dilinde (TR/EN) üretilir. Yeni kurallar RegisterCustomRule() ile runtime'da eklenebilir. NG10Filter, N veya NG etiketli iddiaları otomatik filtreler.	Kural örneği: {name: "engine-gap", condition: "Sadece 1 AI motorunda mention var", action: "Diğer motorlarda görünmek için içerik stratejisi öner"} → Tetiklenme: Acme sadece ChatGPT'de geçiyor, Gemini'de geçmiyor → Öneri (EN): "Acme is not mentioned in Gemini. To improve visibility, consider..." → Öneri (TR): "Acme, Gemini'de hiç geçmiyor. Görünürlük için şu adımları izleyin..." NG10 filtresi: "Acme is the best" (N=NG → filtrele) → "Acme is one of the leading companies" (NG değil → göster)	Veriyle doğrulanmış, sektöre göre kişiselleştirilmiş kurallar

İP-08: Proof of Concept — 5 Algoritma Prototipi

Alan	Detay
Süre	5 hafta
Öncelik	★★★★★ Kritik
Bağımlılık	İP-03, İP-04, İP-05, İP-06, İP-07 (Diğer paketlerin çıktıları)

Ne İsteniyor?

Yukarıdaki tüm araştırma ve modelleme çalışmalarının **çalışan Python prototiplerini** üretmek. Her prototip, gold dataset üzerinde doğrulanmış ve minimum %80 başarı oranına ulaşmış olmalı.

Somut çıktılar:

1. 5 çalışan prototip (Python)

- **Citation Extraction PoC:** AI cevabından citation çıkarma, tür sınıflandırma, URL doğrulama
- **Entity Extraction PoC:** 7 entity tipi ile çıkarım, belirsizlik çözümü
- **Recommendation Detection PoC:** "öneririm/tavsiye ederim/alternatif" tespiti, güç skorlama
- **Prompt Classification PoC:** Intent/topic/persona/funnel sınıflandırma
- **Visibility Score PoC:** 7 bileşenli VI hesaplama, ağırlıklandırma, duyarlılık analizi

2. Performans raporları

- Her PoC için: işlem süresi (ms), bellek (MB), QPS
- 10× veri ölçeklenebilirlik notları
- Darboğaz analizi

3. Doğruluk metrikleri

- accuracy, precision, recall, F1-score, confusion matrix
- Edge case analizi: hangi senaryoda başarısız?
- Minimum kabul: tüm metriklerde > %80

Neden Bu İş Veriliyor?

Bu prototipler, araştırma aşamasından ürünleştirme aşamasına geçişin köprüsüdür. Bir algoritmanın teoride çalışması ile üretimde çalışması arasındaki farkı kapatır. Prototipler onaylandıktan sonra Go backend'e entegrasyon mühendislik ekibi tarafından yapılır.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Citation PoC	Python prototipi, AI cevabından citation'ları çıkarır, türünü sınıflandırır (direct/attribution/directional) ve URL'leri doğrular. Prototip onaylandıktan sonra aynı mantık Go adapter'lara (ChatGPT, Gemini, Perplexity, Claude) taşınır. Her adapter kendi API'sinden gelen citation formatını bu mantıkla ayrıştırır.	Girdi (ChatGPT API yanıtı): { "url_citation": [{ "url": "https://acme.com/hakkimizda" }], "content": "Acme 2005'te kuruldu [kaynak] (https://acme.com)" } → PoC çıktısı: [{url: "https://acme.com/hakkimizda", type: "direct", domain: "acme.com", valid: true}, {url: "https://acme.com", type: "attribution", domain: "acme.com", valid: true}] → URL ziyaret edilir, "2005'te kuruldu" bilgisi teyit edilir.	Citation çıkarma algoritmasının referans implementasyonu
Entity PoC	Python prototipi, AI cevabındaki entity'leri çıkarır ve tiplendirir. Prototip onaylandıktan sonra entity çıkarma mantığı computePresenceShare() ve computeCompetitorContext() fonksiyonlarında kullanılacak Go NER servisine taşınır.	Girdi: "Apple ve Samsung akıllı saat pazarında rekabet ediyor" → PoC çıktısı: [{entity: "Apple", type: "brand", confidence: 0.97}, {entity: "Samsung", type: "competitor", confidence: 0.95}] → Sonra: Go'da extractEntities() çağrılır, aynı çıktı alınır, Presence Share hesaplamasına eklenir.	Entity tanıma yaklaşımının doğrulanması
Recommendation PoC	Python prototipi, ScoreSnapshot ve AuditSnapshot alır, 10+ kuralı değerlendirir, Opportunity	Girdi: ScoreSnapshot: {current: 42, previous: 68} , AuditSnapshot: {robotsDisallowedAll: true}	Öneri tespit mantığının prototipi

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
	Score hesaplar. Prototip onaylandıktan sonra aynı mantık evaluateBrand() fonksiyonunda Go'ya taşınır.	→ PoC çıktısı: [{rule: "score-drop", score: 72, title: "Skor düşüşü tespit edildi", detail: "..."}, {rule: "robots-blocked", score: 30, title: "robots.txt AI bot'larını engelliyor", detail: "..."}] → Go'da evaluateBrand() aynı sonucu üretmeli.	
Prompt Classification PoC	Python prototipi, bir prompt'u intent/topic/persona/funnel olarak sınıflandırır. Prototip onaylandıktan sonra classifyPrompt() fonksiyonu olarak Go'ya taşınır veya Python mikroservisi olarak kalır.	Girdi: "Acme'nin en iyi ürünü hangisi?" → PoC çıktısı: {intent: "recommendation", topic: "product", persona: "consumer", funnel: "decision", confidence: 0.91} → Ölçüm kaydına eklenir, skor hesaplamada intent ağırlığı kullanılır.	Prompt sınıflandırma modelinin referansı
Visibility Score PoC	Python prototipi, 7 bileşenli Visibility Index hesaplar. Prototip onaylandıktan sonra CalculateScore() fonksiyonu Go'da yeniden yazılır. Eski 4 bileşenli hesaplama, yeni 7 bileşenli hesaplama ile değiştirilir.	Girdi: 3 motor cevabı, sentiment: 0.82, NER: 5 mention, citation kalitesi: 0.90 → PoC çıktısı: VI = 0.25×0.80 + 0.20×0.70 + 0.15×0.90 + 0.10×0.60 + 0.10×0.85 + 0.10×0.75 + 0.10×0.95 = 78.5, CI: ±3.5 → Go'da calculateScore() aynı formülü kullanır, aynı sonucu üretir.	Yeni VI algoritmasının prototip implementasyonu

İP-09: Patent Başvurusu Hazırlığı

Alan	Detay
Süre	3 hafta (eşzamanlı, diğer paketlerle birlikte yürür)
Öncelik	★★★★☆ Yüksek
Bağımlılık	İP-03, İP-04, İP-06, İP-08 (Teknik içerik hazır olmalı)

Ne İsteniyor?

Geliştirilen algoritmaların **patentlenebilir yönlerini** belirlemek, prior art taraması yapmak ve patent başvurusuna hazır disclosure dokümanları hazırlamak.

Somut çıktılar:

- Patent Disclosure** (en az 1 adet, başvuruya hazır)
 - Teknik açıklama, yenilik unsuru, mevcut çözümlerden fark
 - Patent vekiline verilecek seviyede detay
- Prior Art Analysis**
 - USPTO, WIPO, Google Patents taraması
 - Her aday için farklılaşma analizi
 - Patent alabilirlik değerlendirmesi
- Patent Adayları Listesi** (en az 3 aday)
 - Cross-source hallucination detection
 - Multi-engine visibility scoring algorithm
 - AI-specific entity disambiguation
 - Opportunity scoring with ML-based impact prediction

Neden Bu İş Veriliyor?

GeoLens'in rekabet avantajını koruması için geliştirilen algoritmaların patentlenmesi gerekir. Algoritma geliştirme sürecinde patentlenebilir yönlerin belirlenmesi ve prior art taramasının yapılması, fikri mülkiyet stratejisinin temelidir.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Patent Disclosure	Geliştirilen her algoritma için patentlenebilir yönler belirlenir ve disclosure dokümanı yazılır. Disclosure, patent vekiline verilir. Vekil bu dokümanla patent başvurusunu hazırlar. Disclosure'ta algorit		

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
	sırası, tahmini maliyet ve başvuru takvimi çıkarılır. Adaylar backlog'a eklenir ve roadmap'e işlenir.	(öncelik: yüksek, maliyet: €5000, süre: 12 ay) 2. Multi-engine visibility scoring algorithm (öncelik: yüksek, maliyet: €5000, süre: 12 ay) 3. AI-specific entity disambiguation in brand contexts (öncelik: orta, maliyet: €4000, süre: 10 ay) Backlog'da: "2026 Q4: Patent başvurusu #1, 2027 Q1: Patent başvurusu #2"	

İP-10: Whitepaper ve Akademik Yayın

Alan	Detay
Süre	4 hafta (eşzamanlı)
Öncelik	★★★★☆ Yüksek
Bağımlılık	İP-06, İP-08 (Teknik sonuçlar hazır olmalı)

Ne İsteniyor?

GeoLens metodolojisini anlatan **akademik kalitede bir whitepaper** ve sektör konferansları için sunum hazırlığı.

Somut çıktılar:

1. Whitepaper v1 (10-15 sayfa)

Sektörde "AI Visibility" dendiğinde akla gelen ilk isim GeoLens olmalı. Whitepaper, konferans sunumları ve blog serisi, düşünce liderliği pozisyonu almak, potansiyel müşteri lead'i üretmek ve organik trafik çekmek için gereklidir.

Çıktı Nerede Kullanılacak?

Çıktı	Yazılımda Nasıl Kullanılır	Örnek	Ne İşe Yarar
Whitepaper	GeoLens'in metodolojisini anlatan akademik doküman. Açık standart olarak herkesin erişimine açıktır. Potansiyel müşterilere "AI Visibility ölçümü nasıl yapılır?" sorusunun cevabı olarak gönderilir. Yatırımcı sunumlarında referans gösterilir.	Whitepaper içindekiler: 1. Giriş — AI çağında marka görünürlüğü sorunu 2. Literatür taraması — 30+ akademik kaynak 3. AI Visibility Framework — metodoloji 4. Visibility Index — matematiksel model 5. Pilot çalışma — 100 marka, 7 motor, 3 sektör 6. Sonuç — GeoLens'in konumu → Satış ekibi müşteriye "Şu whitepaper'ı okuyun, sonra konuşalım" der.	Sektörde düşünce liderliği, açık standart
Konferans Sunumu	BrightonSEO, SearchLove, AI Summit gibi konferanslarda sunulmak üzere hazırlanır. 15-20 slayt, CFP submission özetiyle birlikte teslim edilir. Sunumda GeoLens'in bulguları, metodolojisi ve sektöre katkısı anlatılır. Son slayt demo daveti içerir → lead toplama.	BrightonSEO CFP: Başlık: "AI Visibility: How to Measure Your Brand in Chat	

Hafta 12-14: İP-06 Visibility Index (İP-02+İP-03+İP-05'ten sonra)
Hafta 15-17: İP-07 Opportunity Engine (İP-06'dan sonra)
Hafta 15-19: İP-08 PoC'ler (tüm paketlerden sonra)
Eşzamanlı: İP-09 Patent (İP-03/04/06 yeterli olgunluğa erdiğinde)
Eşzamanlı: İP-10 Whitepaper (İP-06/08 sonuçlarıyla)

5. Değerlendirme Kriterleri

Kriter	Açıklama
Tamamlanma	Tanımlı tüm çıktılar eksiksiz sunulmuş mu?
Doğruluk	Model metrikleri (F1, precision, recall) hedeflenen seviyede mi?
Ürüne Entegrasyon	Çıktılar ilgili kod/doküman dosyalarına işlenmiş mi?
Tekrarlanabilirlik	Başka bir araştırmacı aynı yöntemle aynı sonuçlara ulaşabilir mi?
Patent Potansiyeli	Algoritma patentlenebilir nitelikte mi?
Dokümantasyon	Açıklanabilir ve tekrarlanabilir doküman üretilmiş mi?

6. İletişim ve Raporlama

- Haftalık:** Her Cuma 15:00 — yapılan, yapılacak, blokajlar
- İki haftada bir:** İş paketi ilerleme raporu (teslimat listesine göre % tamamlanma)
- Her iş paketi sonu:** Demo + teslimat teslimi + kod/doküman entegrasyonu